Good vs. Bad MVP Specifications

An interactive worksheet to assess clarity, coherence and readiness.

A specification is useful when it helps a team make consistent decisions without repeatedly guessing what the product should do. This worksheet contrasts weak and stronger examples, then gives you space to assess your own MVP specification.

How to use this worksheet

1. Read each BAD / BETTER comparison and identify the difference.
2. Choose the rating that best describes your current specification.
3. Write a concrete improvement where prompted.
4. Complete the readiness checklist and summarize your next decision.

A strong specification makes the important decisions explicit

It connects the problem and intended user outcomes to Core Jobs, product capabilities, requirements, observable acceptance criteria, learning measures and technical boundaries. It also makes uncertainty and exclusions visible.

Rating scale

Weak / unclear	Important decisions remain vague, assumed or contradictory.
Partly defined	Some decisions are explicit, but gaps remain.
Clear / testable	A teammate can understand, implement and assess the decision.

1 · Problem and primary user

A specification should identify whose problem matters and what meaningful change the product is intended to create.

BAD: broad audience and claim	BETTER: specific user and problem
“This app is for everyone who likes books. It makes finding books easier.”	“Readers who struggle to choose their next book need a more relevant shortlist because general bestseller lists often miss their preferences.”

Self-check 1

Can you identify the primary user, the problem and the context in which it occurs?

- ○ Unclear / broad
- ○ Partly specific
- ○ Specific

One change that would make the problem or user more precise:

2 · Outcomes, Core Jobs and traceability

The specification should show how the product’s work connects to an outcome that matters to the user.

BAD: feature list	BETTER: traceable product logic
“Add ratings, profiles, recommendations, social sharing and notifications.”	“Help the reader choose a book they are likely to enjoy. The shortlist capability supports this Core Job; its success can be assessed with a defined user measure.”

Self-check 2

Can you trace the proposed capabilities back to a user outcome and Core Job?

- ○ No clear link
- ○ Some links
- ○ Traceable

One missing or weak link to strengthen:

3 · Scope and the complete core journey

A good MVP is not simply “everything important, but smaller.” It is the smallest coherent product that can deliver its core promise and test important assumptions.

BAD: everything is a Must Have	BETTER: explicit boundaries
“All requested features are Must Haves. We can decide what to cut during development.”	“Must-Have capabilities support the selected Core Jobs and the end-to-end journey. Should Have, Later and Not in MVP are explicit, with dependencies and reasons for exclusions.”

Self-check 3

Is the MVP boundary explicit, and does the remaining scope still provide a complete path to value?

- ○ Unclear / bloated
- ○ Partly bounded
- ○ Coherent boundary

A feature or scope decision to revisit:

4 · Requirements and acceptance criteria

BAD: vague requirement	BETTER: observable behavior
“Recommendations should be good, fast and personalized.”	“When the user requests recommendations with sufficient input, show a defined number of results and a short explanation for each. If input is insufficient, explain the next step.”

Self-check 4

Could another person implement the key behavior without repeatedly guessing what you mean?

- ○ Vague
- ○ Partly testable
- ○ Testable

A requirement to make more testable:

5 · Learning, evidence and technical boundaries

A useful MVP is designed to learn. The specification should make key assumptions, success signals and material constraints visible.

BAD: launch and hope	BETTER: explicit learning plan
“Build it, launch it and see whether users like it.”	“Test the riskiest assumption with a defined user group. Record the signal that would support or challenge it, and identify technical constraints that could change the solution.”

Self-check 5

Does the plan state what must be learned, how evidence will be gathered and which constraints matter?

○ Not defined ○ Partly defined ○ Explicit

The most important assumption or constraint to clarify:

Readiness checklist

Tick each item only when it is explicit enough to guide a real decision.

- ☐ Problem and primary user are specific.
- ☐ The user context and intended outcome are clear.
- ☐ The product value proposition is understandable.
- ☐ Core Jobs are explicit and connected to capabilities.
- ☐ MVP inclusions and exclusions are justified.
- ☐ The core journey provides a complete path to value.
- ☐ Key requirements have observable acceptance criteria.
- ☐ Risks, assumptions and learning signals are defined.
- ☐ Material technical boundaries and dependencies are visible.
- ☐ Key decisions can be traced back to user outcomes.

Your review summary

Use your ratings and checklist to decide what to improve before treating the specification as ready for implementation.

1. Biggest weakness

Which gap most threatens clarity, coherence or delivery?

2. Decision before development

What must be resolved, tested or explicitly accepted before proceeding?

3. Most important assumption

What assumption matters most, and what evidence would challenge it?

4. Next action and owner

Write one concrete next action and who will take it.

Decision principle

Do not aim for a perfect document. Aim for explicit decisions, visible uncertainty and enough shared clarity to make the next step responsibly.